

# One Cloud for the Entire App

Stakeholder option paper | 19 September 2026 | Version 2.0

**Purpose.** Present a complete single-provider alternative to the previously proposed Netlify, Railway, Supabase and Firebase combination. The frontend is included in the alternative: retaining Netlify would no longer meet the single-provider requirement.

**Primary alternative: AWS.** Host the web app, API, workers, database, cache, identity, storage and AI inference within AWS. Google Cloud and Microsoft Azure are also viable complete alternatives. All require deployment and integration work; none is a one-click move of the current system.

| Decision factor             | AWS                                                  | Google Cloud                                                          | Microsoft Azure                                                            |
|-----------------------------|------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------|
| Entire app on one provider? | Yes, using Bedrock for AI                            | Yes, using Vertex AI for AI                                           | Yes, using Azure-hosted Foundry models                                     |
| Strongest fit               | Team seeking AWS infrastructure and service controls | Team building around Google services and existing Firebase knowledge  | Team already using Microsoft cloud and identity                            |
| Development hosting*        | \$100-250 / month                                    | \$90-250 / month                                                      | \$100-280 / month                                                          |
| Higher-traffic hosting*     | \$500-2,000 / month                                  | \$450-1,800 / month                                                   | \$500-2,200 / month                                                        |
| Migration effort            | High: auth, storage, data and deployment             | Medium-high: data, storage and deployment; assess identity continuity | High: auth, storage, data and deployment                                   |
| Key tradeoff                | Several AWS services to configure and operate        | Persistent data services still cost money when web traffic is idle    | Service tiers, identity integration and model availability need validation |

\*Architecture-level planning allowances, not vendor quotations or validated capacity. Exclude AI inference, tax, engineering and contingency. Definitions and assumptions are on page 3. Cost ranges overlap and should not be treated as a price ranking.

## What "single platform" means

One cloud vendor for application runtime, data, authentication, storage and model access, with cloud charges consolidated through its billing system. It does not mean one server, one product or a fixed subscription. Select models served and billed through that cloud; calling an external model API would introduce another provider.

This document replaces the earlier PDF for the stakeholder's alternatives discussion. It does not change the current project, authorize spending or initiate a migration. Final selection depends on team skills, target region, model suitability and a priced deployment design.

# Complete Service Mapping

Each column is a full alternative. No Netlify, Railway or Supabase account is required for these designs.

| App capability           | AWS alternative                         | Google Cloud alternative                                 | Azure alternative                                               |
|--------------------------|-----------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------|
| Next.js frontend         | ECS on Fargate; CloudFront for delivery | Cloud Run; load balancer / Cloud CDN as needed           | Container Apps; Front Door as needed                            |
| NestJS API               | ECS on Fargate                          | Cloud Run service                                        | Container Apps                                                  |
| Workers / LiteLLM        | Separate ECS services or tasks          | Cloud Run worker pools / jobs; gateway service           | Container Apps with suitable worker scaling / jobs; gateway app |
| Postgres + pgvector      | RDS for PostgreSQL                      | Cloud SQL for PostgreSQL                                 | Azure Database for PostgreSQL Flexible Server                   |
| Redis / compatible cache | ElastiCache for Redis OSS or Valkey     | Memorystore                                              | Azure Managed Redis                                             |
| Customer authentication  | Amazon Cognito                          | Identity Platform / Firebase Authentication under Google | Microsoft Entra External ID                                     |
| Private file storage     | Amazon S3                               | Cloud Storage                                            | Blob Storage                                                    |
| AI model inference       | Amazon Bedrock                          | Vertex AI                                                | Microsoft Foundry models hosted and billed by Azure             |
| Secrets / diagnostics    | Secrets Manager / CloudWatch            | Secret Manager / Cloud Logging and Monitoring            | Key Vault / Azure Monitor                                       |
| Container images         | Amazon ECR                              | Artifact Registry                                        | Azure Container Registry                                        |

Platform and model capabilities: sources [1-10]. Verify regional availability, quotas and pgvector versions before implementation. Redis-compatible engines must be tested against the application's queue library and persistence requirements.

## AWS request flow

Browser loads the frontend hosted on AWS and signs in through Cognito. The API checks user and organization access, reads RDS data and invokes Bedrock through the gateway. ElastiCache supports queued work; ECS workers process documents and write private files to S3.

LiteLLM can remain a self-hosted gateway in the chosen cloud, subject to connector testing. Model access, permissions and request formats change. If the embedding model changes, existing document vectors must be regenerated and retrieval quality tested.

# Development & Growth Costs

USD per month, before tax. AI inference is charged separately even when it appears on the same cloud bill.

**Development:** 5-20 active users, 1,000 AI interactions/month, 1 GB files and 10 GB outbound data. **Growth:** 10,000 active users, 300,000 interactions/month, 100 GB files and 1 TB outbound data. These are planning scenarios, not measured throughput commitments.

Assume one region, small managed data services, containerized web/API/worker/gateway processes and ordinary text/document workloads. No GPU hosting, multi-region failover or enterprise support. Growth allows larger resources and replicas but does not promise high availability. Region, peak concurrency, database size and required recovery targets must be priced before approval.

| AWS hosting allocation                         | Development      | Growth             |
|------------------------------------------------|------------------|--------------------|
| Web, API, workers and gateway                  | \$25-60          | \$150-500          |
| RDS database                                   | \$25-60          | \$100-450          |
| ElastiCache                                    | \$10-30          | \$50-200           |
| File storage / backup allowance                | \$5-15           | \$30-150           |
| Delivery, load balancer and network            | \$20-50          | \$100-450          |
| Logs, registry, secrets and identity allowance | \$15-35          | \$70-250           |
| <b>Hosting subtotal</b>                        | <b>\$100-250</b> | <b>\$500-2,000</b> |

These line items are budget allocations, not quoted service prices. The allowances must be replaced with a region-specific calculator estimate. Persistent managed database/cache services create a baseline cost even with few users. [1-3]

| Hosting + AI example | Development total | Growth total   |
|----------------------|-------------------|----------------|
| AWS                  | \$105-300         | \$2,000-17,000 |
| Google Cloud         | \$95-300          | \$1,950-16,800 |
| Microsoft Azure      | \$105-330         | \$2,000-17,200 |

**AI assumption:** \$0.005-0.05 per completed interaction, giving \$5-50/month in development and \$1,500-15,000/month in growth. These hypothetical rates are sensitivity examples, not prices for a named model. Include all internal calls and retries; embeddings, OCR and other paid tools need separate allowances.

## Proposed AWS pilot envelope

Reserve **\$375/month**: the \$300 upper development example plus 25% contingency. This is a proposed allowance, not approved spend or a guaranteed cap. Enforce application usage limits and measure representative AI tasks before committing a growth budget. Exclude one-time migration labor, domain registration and optional paid tooling.

# Decision & Implementation

Select the cloud first, then validate a priced pilot before committing to production.

| Step                            | Outcome needed                                                                                                                                         |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Choose provider              | AWS is the lead alternative. Prefer Google Cloud or Azure when existing skills, contracts or model requirements favor them. Confirm deployment region. |
| 2. Validate models and identity | Verify chat and embedding model quality, access and quotas. Plan identity migration and account mapping; do not assume existing passwords transfer.    |
| 3. Build the cloud deployment   | Containerize web/API/workers; configure networking, secrets, domain/TLS and service identities. Migrate Postgres and files with validation.            |
| 4. Test the pilot               | Test tenant access, streaming, uploads, vector search, queues and restores. Measure tokens, latency, concurrency and monthly run rate.                 |
| 5. Approve and release          | Sponsor approves measured budget; technical owner signs off recovery and rollback. Switch traffic only after acceptance.                               |

**Ownership:** sponsor selects budget and provider; engineering owns migration and operations; finance validates cloud billing and local-currency treatment. No deployment date or migration fee is committed. No external resources were created for this document.

## Official references

Reviewed 19 September 2026. Sources establish service capability and pricing methodology; workload estimates are independent planning judgments. Consult current calculators and account terms for a purchase decision.

- [1] [AWS Fargate pricing](#)
- [2] [Amazon RDS PostgreSQL pricing](#)
- [3] [Amazon Cognito pricing](#)
- [4] [Bedrock supported models and regions](#)
- [5] [Cloud Run services and integrations](#)
- [6] [Cloud Run pricing](#)
- [7] [Cloud SQL vector search](#)
- [8] [Azure Container Apps pricing](#)
- [9] [Azure Managed Redis pricing](#)
- [10] [Microsoft Foundry model hosting](#)
- [11] [Entra External ID](#)
- [12] [Azure PostgreSQL pgvector](#)